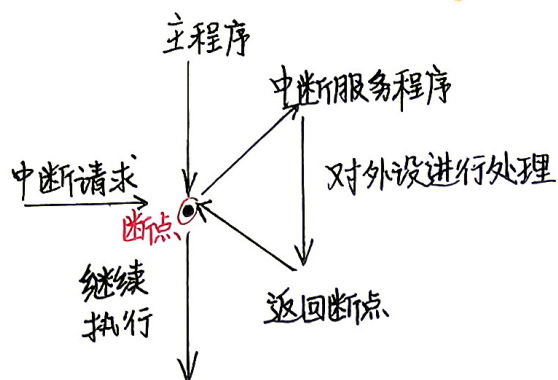


第6章



中断的定义:

CPU在执行程序中,被内部或外部事件所打断,转去执行一段预先安排好的中断服务程序;服务结束后返回原断点,继续执行原程序

中断的处理过程: 中断请求 → 中断响应 → 中断服务 → 中断返回

进行数据交换的
实质性环节

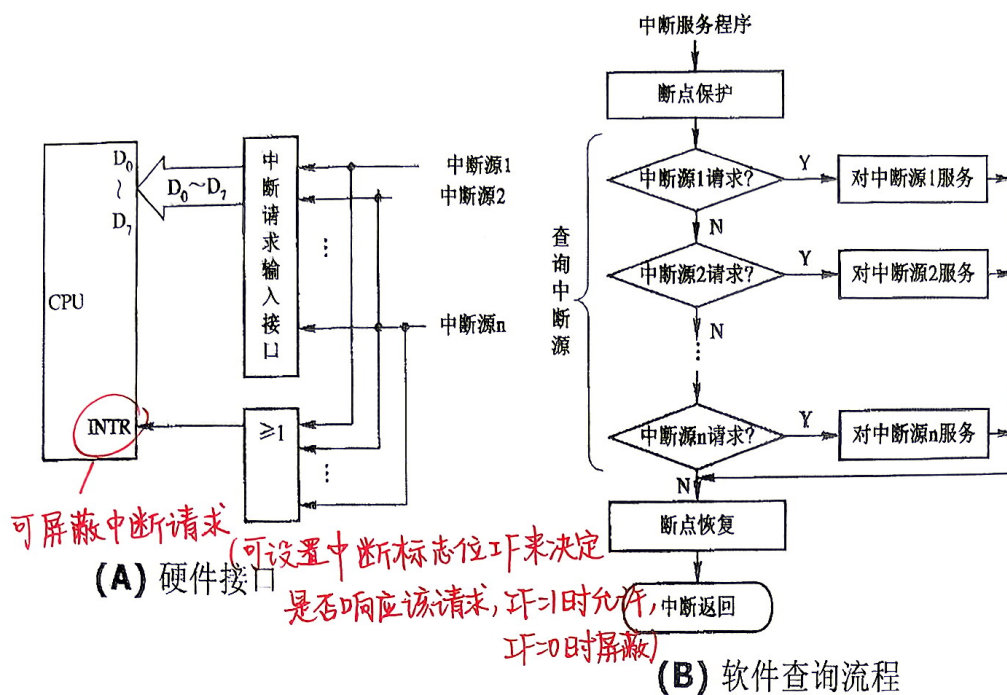
系统有多个中断请求, CPU如何识别中断源?

- ① 向量中断
- ② 中断查询

如有多个中断同时请求, CPU如何应对?

- ① 软件查询
- ② 链式优先权排队电路
- ③ 优先权编码电路

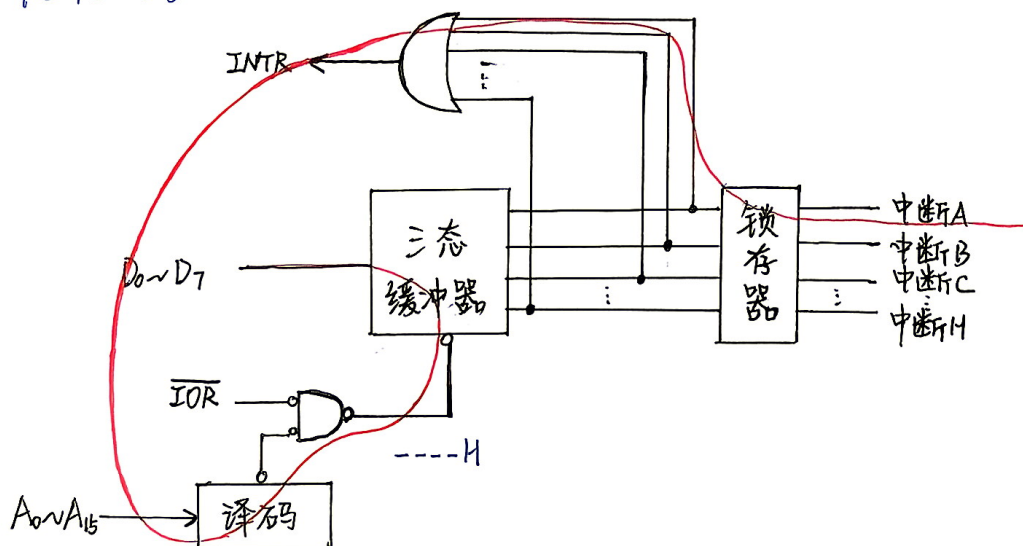
中断处理过程中又有中断提出请求(中断嵌套), 如何应对?



① 向量中断:

CPU 响应中断时, 发出中断响应信号, 由中断控制器 (例如 8259A) 送出中断类型号, CPU 根据中断类型号, 以中断向量表中找到中断服务程序的入口地址, 该中断称为向量中断。由于采用硬件结构提供中断向量, CPU 无需花时间查询状态位, 中断响应速度快。

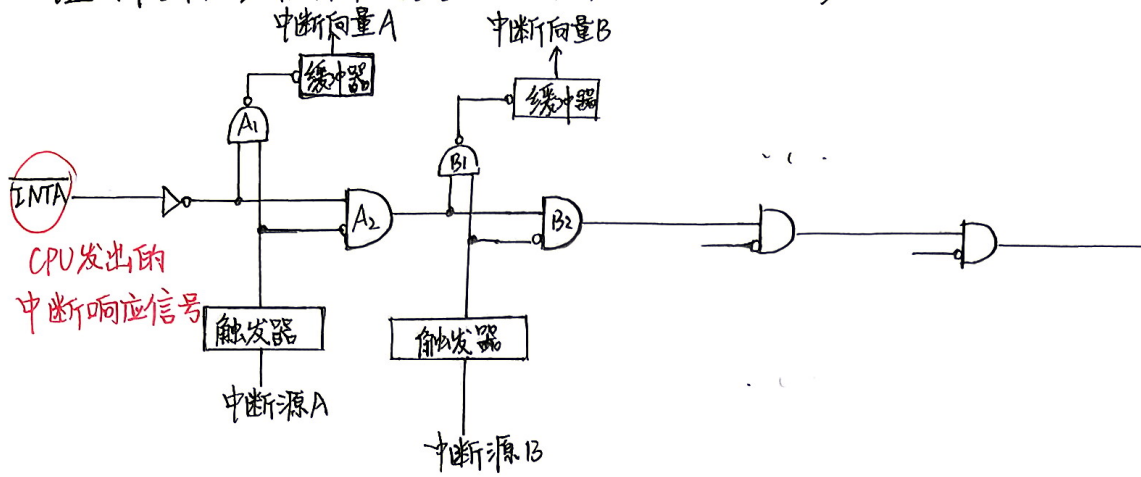
② 中断查询接口:



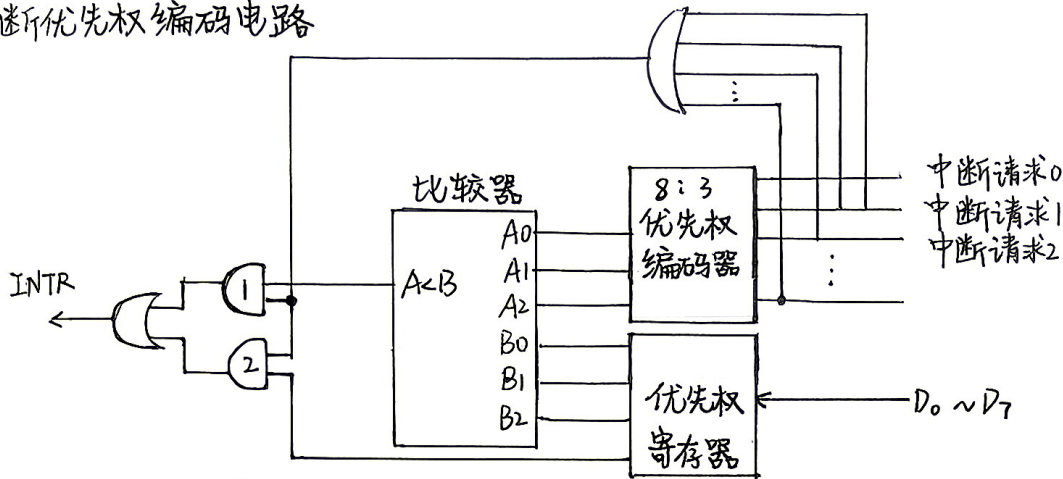
工作逻辑:

外部设备的中断请求先被锁存器暂存状态, 只要有任意设备发送请求, 锁存器的输出经“或门”触发 CPU 的 INTR 中断信号。待 CPU 开中断 (IF=1) 并响应后, 会通过地址总线发送 4 位 16 进制数的接口地址, 配合 $\overline{IOR}$ 读信号, 经译码电路选通三态缓冲器, 最终从数据总线读取锁存器中设备请求状态, 以确定具体中断源并执行对应服务程序。

链式中断优先级排队电路 (设优先级 $A > B > \dots$)



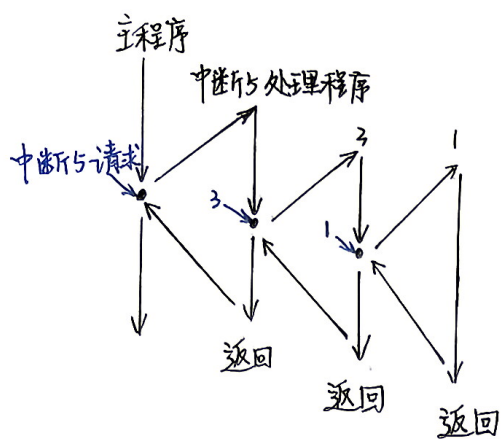
中断优先级编码电路



多中断源发起请求后, 8:3 优先级编码器先把其中优先级最高的请求转换成3位编码, 送到比较器的A端

同时将CPU当前正在处理的中断优先级编码存在寄存器中, 送到比较器的B端比较A和B的编码大小, 若A的 < B的, 则说明A优先级更高, 触发CPU的INTR中断, 让新请求“插队”处理, 若A优先级更低, 则不触发INTR, 只能等当前中断结束

中断嵌套



如何保证多级嵌套顺利进行?

在中断处理程序中要有开中断指令,大多数微机在响应中断时硬件自动关中断
中断处理程序是在关中断的情况下运行的

对可屏蔽中断而言,若要实现中断嵌套,则务必使中断处理程序在允许中断的情况下

中断工作原理(中断请求→中断响应)

- ① 外设向中断接口电路发送中断请求
- ② 中断接口电路将经过优先级排队的中断请求信号发给CPU
- ③ CPU在 $IF=1$ 时向中断接口电路发送中断响应信号
- ④ 中断接口电路向CPU发送中断类型码,据此CPU可计算中断服务程序的存放地址
- ⑤ CPU停止正在执行的程序,转去执行中断服务程序,在中断服务程序的控制下, CPU与外设进行数据传送

8086/8088 的中断系统采用向量中断机制, 能处理 256 个中断 (0~255)

可屏蔽中断还需借助专用中断控制器 Intel 8259A 实现优先权管理

中断向量: 中断服务程序的入口地址 - 首地址

逻辑地址 - 段地址CS, 偏移地址IP (32位)

每个中断向量的低字是偏移地址,

高字是段地址,

要占用 4 个字节 向量号 N 的中断向量的物理地址: $4N$

256 个中断占用 $256 \times 4B = 1KB$ 区域

形成中断向量表

8088 微处理器从物理地址 000H 开始,

依次安排各中断向量, 向量号从 0 开始

中断向量并非常驻内存, 而开机上电时由程序装入指定存储区

BIOS 程序只负责中断类型号 00H~1FH 共 32 种中断的中断向量装入。

用户若想使用软件中断或编写新的中断服务程序代替旧的中断服务程序, 就要将新的中断服务程序的入口地址装入中断向量指针指定的中断向量表

例. 设置新中断向量 80

```
PUSH DS
MOV DX, OFFSET NEW80H
MOV AX, SEG NEW80H
MOV DS, AX
MOV AX, 2580H
INT 21H
POP DS
```

→ 取中断程序偏移地址

DS: DX = 中断向量 (段地址: 偏移地址)

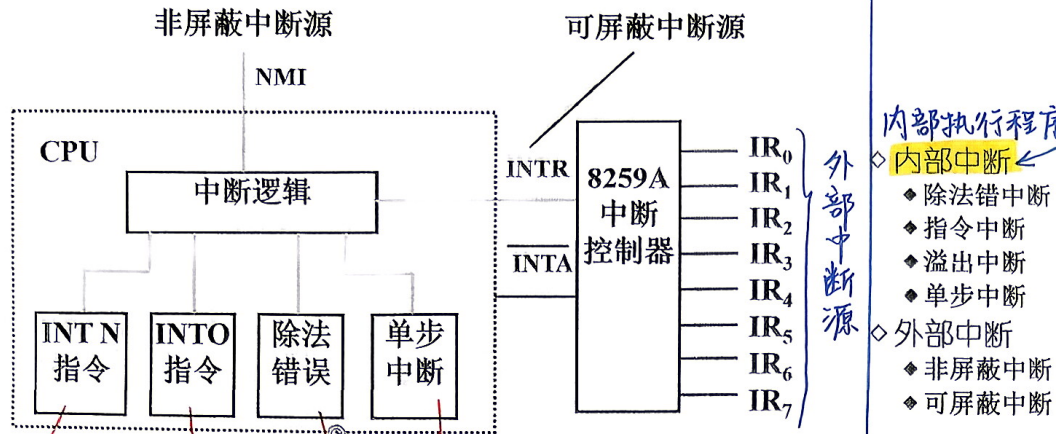
→ 取中断程序段地址

; 功能号: AH=25H, 入口参数: AL=中断向量号

; 设置中断向量 (DOS 功能调用 INT 21H)

用户中断, 向量号: 255

8086/8088 中断系统结构



程序员主动用 溢出中断 除法时除数为0或结果溢出
(OF=1时触发)

TF=1时, CPU 每执行一条指令就触发一次(调试用)

内部中断

(1) 除法错中断 (向量号为 0 的内部中断)

```
MOV BL, 0
IDIV BL
```

```
MOV AX, 200H
MOV BL, 1
DIV BL
```

关于 DIV 和 IDIV:

- DIV 处理无符号数, 余数恒为非负; IDIV 处理有符号数, 余数符号与被除数符号相同
- 16位除法: 被除数从 AX 中取, 除数从寄存器/内存单元(16位)中取,
("16" ÷ "8") 商存到 AL 寄存器, 余数存到 AH 寄存器

32位除法: 被除数的高16位从 DX 中取, 低16位从 AX 中取, 除数从寄存器/内存单元(16位)中取,
("32" ÷ "16") 商存到 AX 寄存器, 余数存到 DX 寄存器

(2) 指令中断 (向量号为 0~255 的内部中断)

执行中断调用指令 INT N 时产生

其中, 向量为 3 的指令中断比较特别, 生成一个字节的指令代码: 11001100,

常用于程序调试, 被称为断点中断 (DEBUG.EXE 调试程序的运行命令 G 设置的断点就是通过 INT 3 指令实现)

(3) 溢出中断 (向量号为 4 的内部中断)

执行溢出中断指令 INTO 时, 溢出标志 OF 为 1 时产生

```
MOV AX, 2000H
```

```
ADD AX, 7000H ; OF=1
```

```
INTO
```

; 产生溢出中断

(4) 单步中断 (向量号为 1 的内部中断)

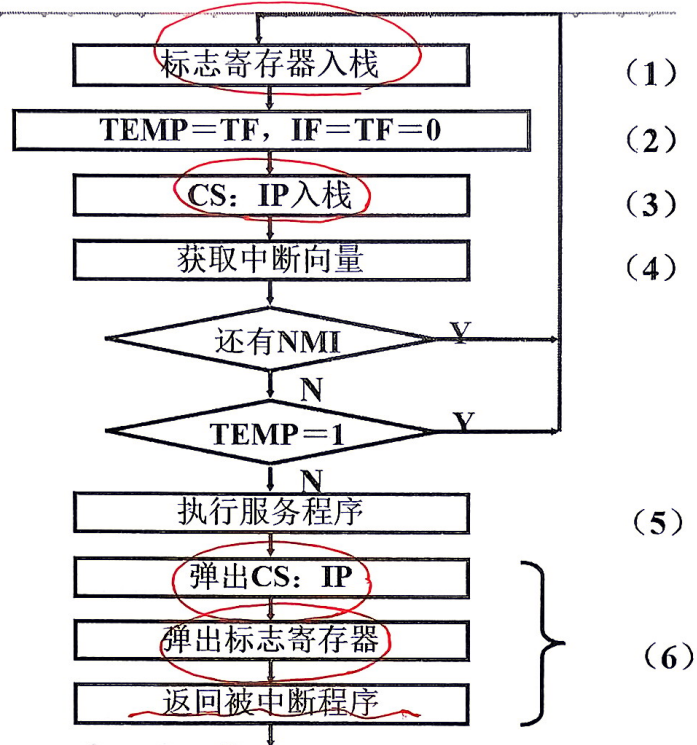
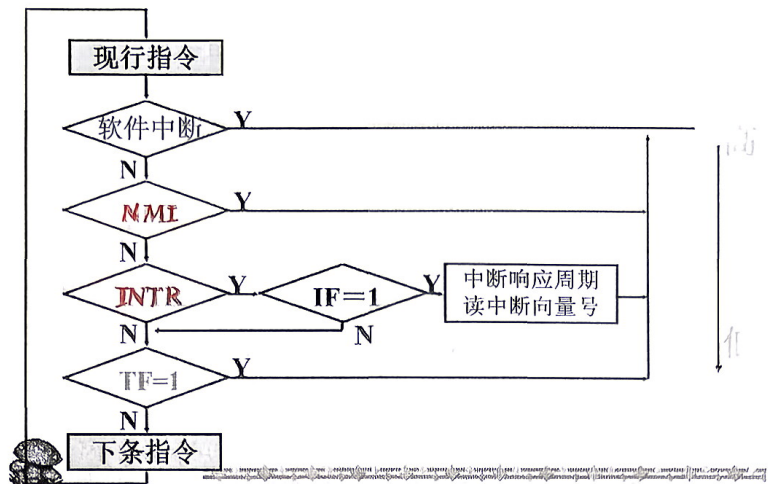
TF 为 1 时, 每条指令执行结束后产生一次

(DEBUG.EXE 调试程序的单步命令 T 利用单步中断实现对程序的单步调试)

查询中断的顺序，
决定了各种中断源的优先权

- ◇ 软件中断
- ◆ 除法错中断
- ◆ 指令中断
- ◆ 溢出中断
- ◇ 非屏蔽中断
- ◇ 可屏蔽中断
- ◇ 单步中断

高
↓
低



- ◇ 8088各种中断源的优先权，
实际上是指被识别出来的先后
- ◇ 多种中断同时请求时，
最先响应的则可能是
单步中断或NMI中断

外部中断

是由外部提出中断请求引起的程序中断

外部中断才是“真正的中断”，是处理器外部随机产生的，

而内部中断是处理器执行程序出现异常引起的，其实只能叫“异常”。

(1) 非屏蔽中断

微处理器无法禁止，将在当前指令结束响应的中断

8088 的非屏蔽中断的向量号是 2，请求信号是 NMI

主要用于处理系统的意外或故障，例如：电源掉电前的数据保护、存储器读写错误的处理

(2) 可屏蔽中断

外部通过可屏蔽中断请求信号向微处理器提出，微处理器在允许可屏蔽中断的条件下，在当前指令执行结束予以响应，同时输出可屏蔽中断响应信号

8088 的可屏蔽中断请求和响应信号分别是 INTR 和 $\overline{INTA}$ ，

由 IF 标志控制可屏蔽中断是否允许响应，向量号来自外部中断控制器

8088 通常要配合 中断控制器 8259A 共同处理可屏蔽中断

主要用于 主机与外设交换数据

IF

IF=0: 可屏蔽中断不被响应

- 关中断、禁止中断、中断屏蔽
- 系统复位，使 IF=0
- 任一中断被响应，使 IF=0
- 执行关中断指令 CLI，使 IF=0

IF=1: 可屏蔽中断被响应

- 开中断、允许中断、中断开放
- 执行开中断指令 STI，使 IF=1

IRET

中断返回指令，恢复原 IF 状态

- 从堆栈中弹出之前保存的 IP、CS
- 从堆栈中弹出之前保存的标志寄存器 FLAGS，其中正包含 IF 原状态
- 跳回原程序继续执行

8259A 中断控制器

Intel 8259A 是可编程中断控制器 PIC (Programmable Interrupt Controller)
用于管理可屏蔽中断

基本功能:

1 片 8259A 可管理 8 级中断, 可扩展至 64 级,

每级中断都可单独被屏蔽或允许。

在中断响应周期, 可提供相应的中断向量号

8259A 有多种工作方式, 可通过编程选择

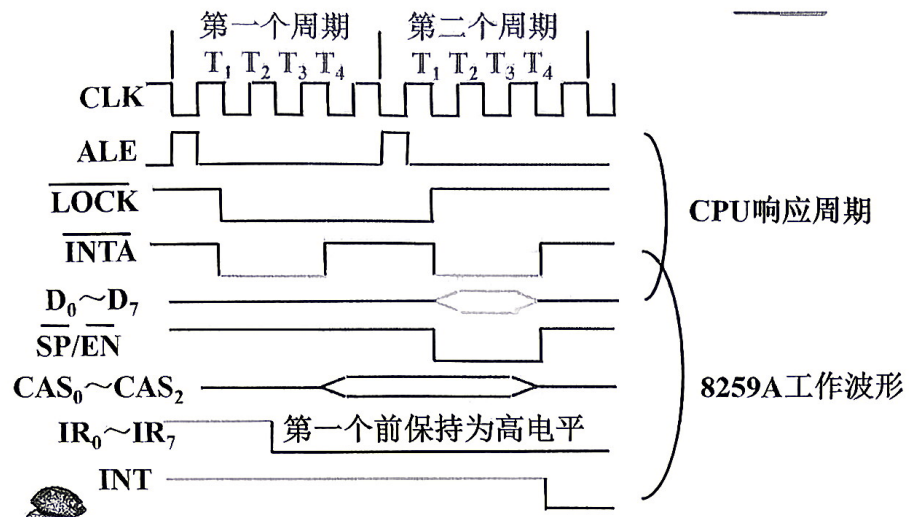
与处理器接口

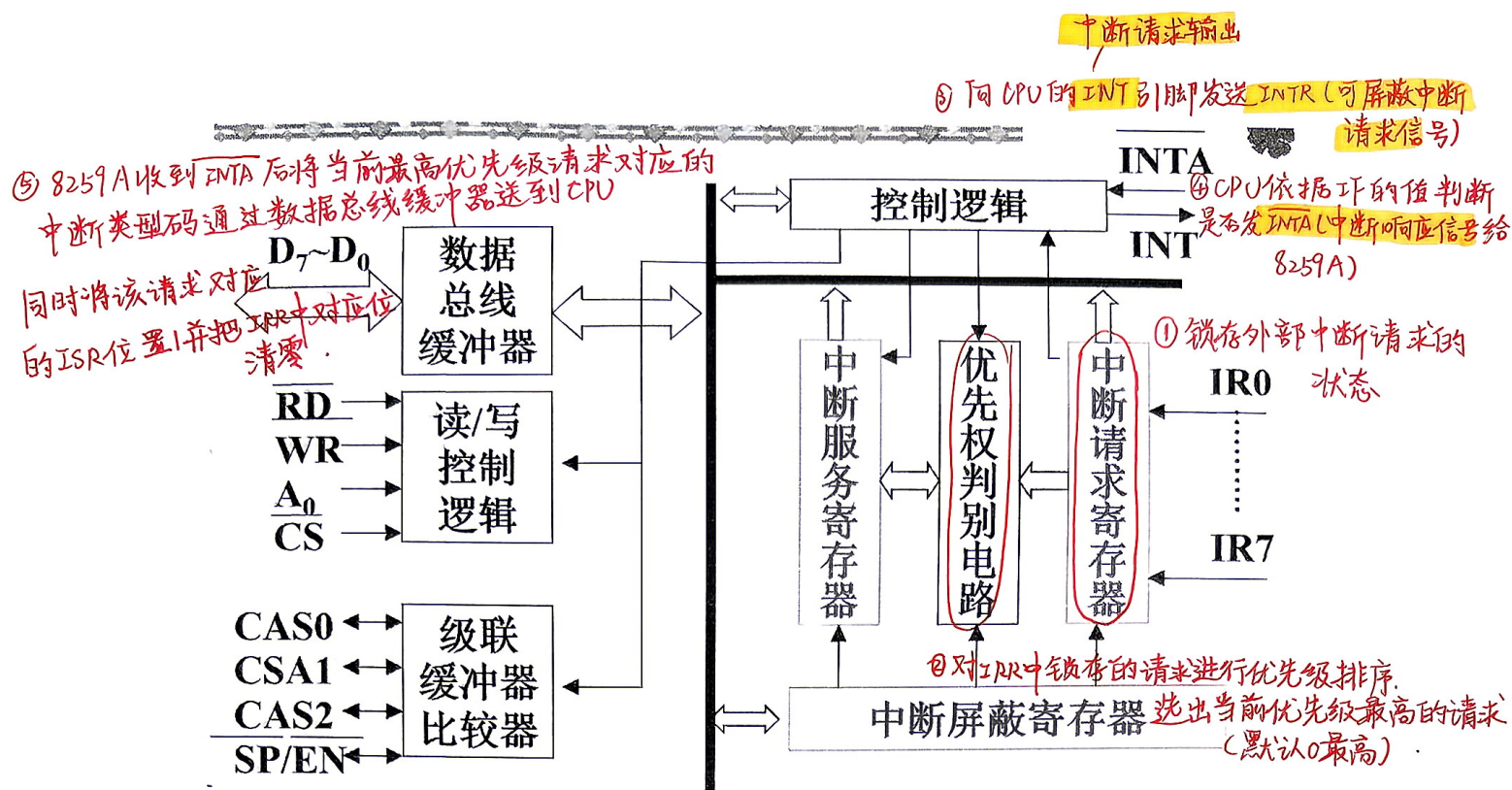
A_0	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	功能
0	1	0	0	写入 ICW1、OCW2、OCW3
1	1	0	0	写入 ICW2、ICW4、OCW1
0	0	1	0	读出 IRR、ISR、查询字
1	0	1	0	读出 IMR
X	1	1	0	数据总线高阻状态
X	X	X	1	数据总线高阻状态

★ 8259A 的一个突出优点就是可以方便地与 CPU 配合实现向量中断, 避免对中断

设备逐个查询

- ① 当 $IR_0 \sim IR_7$ 中有一个或几个中断源变成高电平时, 使相应的 IRR 位置位
- ② 8259A 对 IRR 和 IMR 提供的情况进行分析处理,
如该中断请求唯一或请求的中断比正在处理的优先级高或虽然优先级低但前一个中断服务刚好结束, 就从 INT 端输出高电平, 向 CPU 发出中断请求
- ③ CPU 在每个指令的最后一个时钟周期检查 INT 输入端的状态, 当 IF 为 "1" 且无其它高优先级 (如 NMI) 的中断时, 就响应这个中断, 从此 CPU 进入 2 个中断响应 ($\overline{INTA}$) 周期
- ④ 在 CPU 的第 1 个 $\overline{INTA}$ 周期中, 8259A 接收第 1 个 $\overline{INTA}$ 信号时, 将 ISR 中当前请求中断中优先级最高的相应位置位, 而对应的 IRR 位置复位为 0
- ⑤ 在 CPU 的第 2 个 $\overline{INTA}$ 周期中, 8259A 收到第 2 个 $\overline{INTA}$ 信号时, 送出中断类型号, 实现向量中断





(1) 中断请求寄存器 IRR

保存 8 条外界中断请求信号 $IR_0 \sim IR_7$ 的请求状态

D_i 位为 1 表示 IR_i 引脚有中断请求，为 0 则表示没有

(2) 中断服务寄存器 ISR

保存正在被 8259A 服务的中断状态

D_i 位为 1 表示 IR_i 中断正在服务中，为 0 则表示没有

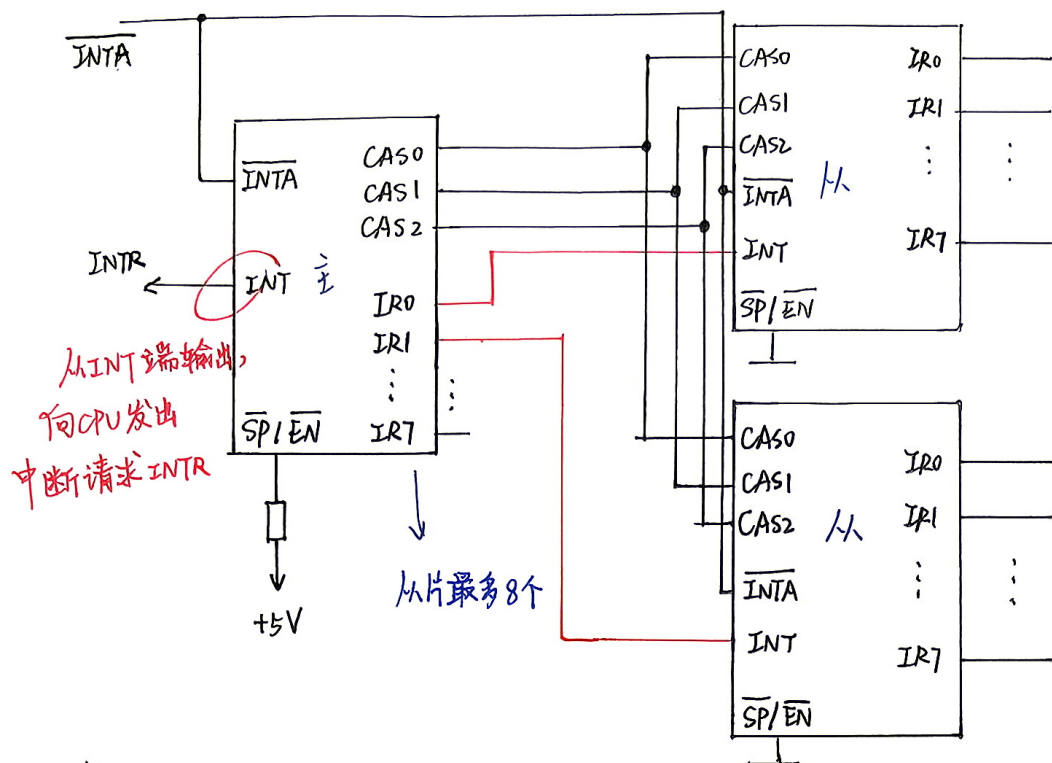
(3) 中断屏蔽寄存器 IMR

保存对中断请求信号 IR 的屏蔽状态

D_i 位为 1 表示 IR_i 中断被屏蔽 (禁止)，为 0 则表示允许

中断级联

8259 A 级联工作示意图



- 每个从8259A的中断请求信号(INT)连至主8259A的一个中断请求输入端(IR_i)
- 主8259A的INT线连至CPU的中断请求输入端(INTR)
- SP1EN是在非缓冲方式下,规定该8259A是主片($\overline{SP1EN}=1$)还是从片($\overline{SP1EN}=0$)

8259A的工作方式

设置优先级方式 { 优先级固定方式 { 普通全嵌套方式
特殊全嵌套方式
优先级循环方式 { 自动循环方式
特殊循环方式

结束中断处理方式 { 自动中断结束方式
非自动中断结束方式 { 普通中断结束方式
特殊中断结束方式

屏蔽中断源方式 { 普通屏蔽方式
特殊屏蔽方式

中断触发方式 { 边沿触发方式
电平触发方式

数据线连接方式 { 缓冲方式
非缓冲方式

1. 设置优先权方式

• 普通全嵌套方式

8259A 的中断优先权顺序固定不变，从高到低依次为 IR0、IR1、IR2、.....IR7

中断请求后，8259A 对当前请求中断中优先权最高的中断 IR_i 予以响应，将其向量号送上数据总线，对应 ISR 的 Di 位置位，至到中断结束（ISR 的 Di 位置位）

在 ISR 的 Di 位置位期间，禁止再发生同级和低级优先权的中断，但允许高级优先权中断的嵌套

• 特殊全嵌套方式

特殊全嵌套方式和全嵌套方式基本相同。唯一不同的地方在于：在特殊全嵌套方式下，当处理某一级中断时，如果有同级的中断请求，那么它也会给予响应，从而实现一种对同级中断请求的特殊嵌套。特殊全嵌套方式一般是用于 8259A 的级联情况下。

• 优先权自动循环方式

规定 IR0 为最高优先权，IR7 为最低。

当某一个中断源受到中断服务后，它的优先权就自动降为最低，而与之相邻的优先级就升为最高。

• 优先权特殊循环方式

优先权特殊循环方式与优先权自动循环方式相比仅有一点不同，就是在优先权特殊循环方式下，一开始的最低优先权是由编程确定的，而不是自动规定，从而也就确定了最高优先权。

2. 结束中断处理方式

8259A 利用中断服务寄存器 ISR 判断：

某位为 1，表示正在进行中断服务；

该位为 0，就是该中断结束服务。

} 如何判断中断结束？

(此处仅反映 8259A 认为该中断服务已完成，进行自身内部寄存器的状态更新，但不能反映 CPU 的工作状态，是否完成中断服务程序的执行)

• 自动中断结束方式

在这种方式下，系统一旦进入中断响应，8259A 就在最后一个中断响应周期的信号的后沿，自动将 ISR 中被响应中断级的对应位清“0”，这是一种最简单的中断结束处理方式，可以通过初始化命令字来设定。

→ ISP

指令 IRET 来体现

• 普通中断结束方式

配合全嵌套优先权方式使用

当 CPU 用输出指令往 8259A 发出普通中断结束 EOI 命令时，8259A 就会把所有正在服务的中断中优先权最高的 ISR 位置位

• 特殊中断结束方式

配合循环优先权方式使用

CPU 在程序中向 8259A 发送一条特殊中断结束命令，这个命令中指出了要清除哪个 ISR 位

3. 屏蔽中断源方式

• 普通屏蔽方式

将 IMR 的 Di 位置 1，则对应的中断 IR_i 被屏蔽，该中断请求不能从 8259A 送到 CPU

如果 IMR 的 Di 位置 0，则允许 IR_i 中断产生

• 特殊屏蔽方式

将 IMR 的 Di 位置 1，对应的中断 IR_i 被屏蔽的同时，使 ISR 的 Di 位置 0

特殊屏蔽方式是用于这样一种特殊要求的场合，即在执行较高级的中断服务时，希望开放较低级的中断请求。

4. 中断触发方式

• 边沿触发方式

8259A 将中断请求输入端出现的上升沿作为中断请求信号

• 电平触发方式

中断请求端出现的高电平是有效的中断请求信号

5. 数据线连接方式

• 缓冲方式

8259A 的数据线需加缓冲器予以驱动

8259A 把 SP*/EN* 引脚作为输出端，输出允许信号，用以锁存或开启缓冲器

• 非缓冲方式

SP*/EN* 引脚为输入端

若 8259A 级连，由其确定是主片或从片

8259A的控制字及中断操作功能

(1) 初始化编程

8259A开始工作前,必须进行初始化编程

给8259A写入初始化命令字 ICW

初始化命令字 ICW

初始化命令字 ICW 最多有 4 个, 8259A 在开始工作前必须写入且必须按照 ICW1 ~ ICW4 的顺序
ICW1、ICW2 必须送, ICW3、ICW4 由工作方式决定

ICW1

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
x	x	x	1	LTIM	x	SNGL	IC4

x 表示任意

1, 0 都可, 建议为 0

D₃: 中断触发方式
1: 电平
0: 边沿

1: 单片
0: 级联

→ 决定是否写入 ICW3

决定是否写入 ICW4

1: 要写入 ICW4

0: 不写入 ICW4, 其规定的位是 0

ICW2

设置中断向量号

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
T ₇	T ₆	T ₅	T ₄	T ₃	x	x	x

高 5 位:

T₇ ~ T₃ 是中断向量号的高 5 位, 8259A 自动确定 (IR₀: 000 ... IR₇: 111)

低 3 位:

ICW3

级联命令字

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
S ₇	S ₆	S ₅	S ₄	S ₃	S ₂ /ID ₂	S ₁ /ID ₁	S ₀ /ID ₀

S_i = 1 说明主片对应的 IR_i 接有从片

ID₀ ~ ID₂ 编码说明从片接到主片哪个引脚 (3 位可表示 IR₀ ~ IR₇ 8 个, 正好)

($\begin{matrix} D_2 & D_1 & D_0 \\ 1 & 0 & 0 \end{matrix} \rightarrow \text{引脚 IR}_4$)

ICW4

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
0	0	0	SFNM	BUF	M/S	AEOL	MPM

1: 特殊全嵌套方式
0: 普通全嵌套方式

1: 缓冲方式
0: 非缓冲方式

1: 自动中断结束
0: 非自动中断结束

D₄: 嵌套方式 D₃: 数据线缓冲方式 D₂: 主从片选择 D₁: 中断结束方式 D₀: 微处理器类型

初始化 8259A

ICW1 固定写入 8259A 的“端口 0” (主片: 20H, 从片: 0A0H)

ICW2 ~ ICW4 固定写入 8259A 的“端口 1” (主片: 21H, 从片: 0A1H)

主片

```
MOV AL, 11H ; 写入 ICW1, 级联、边沿触发
OUT 20H, AL ; 主片 I/O 地址是 20H 和 21H
JMP INTR1 ; 该转移指令起延时作用, 等待 8259A 操作结束
```

INTR1:

```
MOV AL, 08H ; 写入 ICW2, 设主片 IR0 的中断向量号是 08H
OUT 21H, AL
JMP INTR2
```

INTR2:

```
MOV AL, 04H ; 写入 ICW3, 设主片 IR2 级联从片
OUT 21H, AL
JMP INTR3
```

INTR3:

```
MOV AL, 01H ; 写入 ICW4, 设普通全嵌套方式、非自动中断结束方式
OUT 21H, AL ; 这里主从片那里写 0 (是从片) 没关系, 因为仅在 BUF=1 (缓冲方式) 下才区分主从片
```

从片

```
MOV AL, 11H ; 写入 ICW1, 级联、边沿触发
OUT 0A0H, AL
JMP INTR5
```

INTR5:

```
MOV AL, 70H ; 写入 ICW2, 设从片 IR0 的中断向量号是 70H
OUT 0A1H, AL
JMP INTR6
```

INTR6:

```
MOV AL, 02H ; 写入 ICW3, 从片接到主片的 IR2 引脚
OUT 0A1H, AL
JMP INTR7
```

INTR7:

```
MOV AL, 01H ; 写入 ICW4, 设普通全嵌套方式、非自动中断结束方式
OUT 0A1H, AL
```

(2) 中断操作功能

在8259A工作期间,可以写入**操作命令字OCW**将选定的操作传送给8259A,使之按新要求工作,也可以读取8259A的信息,以便了解其工作状态

操作命令字 OCW

8259A工作期间,可随时接受操作命令字OCW

OCW共有3个: OCW1 ~ OCW3

写入时没有顺序要求,要哪个就写哪个

OCW1

屏蔽命令字

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
M ₇	M ₆	M ₅	M ₄	M ₃	M ₂	M ₁	M ₀

内容写入中断寄存器IMR

$D_i = M_i$ 对应 IR_i , 为1表示禁止 IR_i 中断, 为0表示允许 IR_i 中断, 各位相互独立

OCW2

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
R	SL	EOI	0	0	L ₂	L ₁	L ₀

R, SL, EOI 配合使用, 产生中断结束EOI命令和改变优先权顺序

L₂~L₀的3位编码指定IR引脚

OCW3

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
0	ESMM	SMM	0	1	P	RR	RIS

ESMM, SMM 设置中断屏蔽方式

P, RR, RIS 规定随后读取的状态字含义

RR=0: 不读取状态寄存器

RR=1: 允许读取状态寄存器

RIS=0: 读取IRR

RIS=1: 读取ISR

读取状态字

CPU可读出IRR, ISR, IMR和查询字

A₀为低, 由OCW3中RR和RIS位设定读取IRR或ISR, 由OCW3中P位设定读取查询字

A₀为高时读取的都是IMR

查询字反映8259A是否有中断请求

查询字

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
I	-	-	-	-	W ₂	W ₁	W ₀

中断位I位为1表示:

有外设请求中断

W₂~W₀的编码

当前中断请求的最高优先级



A₀电平

对应8259A端口名称

可写入的命令字

可读取的状态寄存器

低电平(0)

端口0(偶端口)

ICW1, OCW2, OCW3

IRR(中断请求寄存器), ISR(中断服务寄存器), 查询字

高电平(1)

端口1(奇端口)

ICW2, ICW3, ICW4, OCW1

IMR(中断屏蔽寄存器)

命令字与状态字的区别方法

① 利用读写信号区分

8259A 通过 CPU 的 $\overline{RD}$ 和 $\overline{WR}$ 来判断操作类型:

若 $\overline{WR}$ = 低电平, 则 CPU 在向 8259A 写入命令字 (如 ICW、OCW)

若 $\overline{RD}$ = 低电平, 则 CPU 在从 8259A 读取状态字 (如 IRR、ISR)

② 利用地址信号区分

通过 A₀ 引脚的电平 (对应不同的 I/O 地址) 来区分要访问的寄存器:

若 A₀ = 低电平 (对应端口 0, 如主片 201H), 则访问的是 ICW1、OCW2、OCW3 等寄存器

若 A₀ = 高电平 (对应端口 1, 如主片 211H), 则访问的是 ICW2、ICW3、ICW4、OCW1 等寄存器

③ 利用控制字的标志位区分

例如 ICW1 的 D₄ = 1 (标志位), 8259A 收到会识别这是 ICW1

例如 OCW2 的 D₃、D₄ = 0, 8259A 会识别这是 OCW2

④ 利用芯片内部的顺序逻辑区分

8259A 内部有顺序控制逻辑, 会按固定顺序识别命令字

初始化阶段, 8259A 会按 "ICW1 → ICW2 → ICW3 → ICW4" 的顺序接收命令字

运行阶段, 会按操作的先后顺序识别 OCW

⑤ 利用前面的控制字决定后续寄存器

先写入的控制字会决定后续操作的寄存器

例如先写入的 ICW1 中指定级联方式, 后续 8259A 会自动识别需要写入 ICW3 (级联配置)

例如先写入 OCW3 中指定读取 ISR, 后续读操作返回 ISR 的状态

注意:

- 利用上升沿做为中断请求IRQ的有效信号
- IRQ0~IRQ7的中断向量号依次为08H~0FH, IRQ8~IRQ15依次为70H~7FH
- 采用普通全嵌套优先权方式, 中断优先权从高到低顺序为IRQ0~IRQ2, IRQ8~IRQ15, IRQ3~IRQ7且不能改变
- 采用普通中断结束EOI方式, 需要在中断服务最后发送普通EOI命令
- 一般采用普通屏蔽方式, 通过写入IMR允许中断, 但注意不要破坏原屏蔽状态

编写主程序:

- 1) 修改中断向量
- 2) 控制CPU的中断允许标志
- 3) 设置8259A的中断屏蔽寄存器

编写外部可屏蔽中断服务程序:

- 1) 发送中断结束命令
- 2) 一般只能采用存储单元传递参数
- 3) 不要使用DOS系统功能调用
- 4) 尽量短小

例. 可屏蔽中断服务程序

8259A的IRQ0(向量号为08H)中断请求来自定时器8253, 每隔55ms产生一次

本程序的08H号中断服务程序, 每次中断显示一串信息, 显示10次

用内存单元(共享变量)在主程序与外部中断服务程序之间传递参数: 中断次数
显示信息也安排在共同数据段中

数据段

DATA SEGMENT

INTMSG DB 'A 8259A Interrupt!', 0DH, 0AH, 0

COUNTER DB 0

DATA ENDS

主程序

MOV AX, 3508H ; AH=35H, AL=08H
INT 21H ; 获取, 返回, 段, 基
PUSH BX ; 原中断向量地址=ES:BX
PUSH ES
CLI ; 关中断, 防止设置新中断向量时发生中断
PUSH DS ; "保护"
MOV AX, SEG NEW08H ; 获取新中断服务程序的段地址
MOV DS, AX
MOV DX, OFFSET NEW08H ; 获取偏移地址
MOV AX, 2508H ; AH=25H, AL=08H
INT 21H ; 将DS:DX设为新中断向量
POP DS ; "恢复"
IN AL, 21H ; 从IMR端口读取当前中断屏蔽寄存器的值
PUSH AX
AND AL, 0FEH ; 允许IRQ0中断
OUT 21H, AL ; 修改后的IMR值写回端口
MOV COUNTER, 0
STI ; 开中断
START1:
CMP COUNTER, 10
JB START1
CLI ; 关中断
POP AX
OUT 21H, AL ; 将原IMR值写回端口21H, 恢复原中断屏蔽设置
POP DX
POP DS
MOV AX, 2508H ; 设置, 恢复原来的08H中断向量
INT 21H
STI ; 开中断
MOV AX, 4C00H
INT 21H ; 返回DOS

中断服务程序

显示字符串子程序

NEW08H PROC

嵌套

STI ; 开中断, 允许中断处理期间响应更高优先级中断

PUSH AX

PUSH BX

PUSH DS

MOV AX, @data ; 获取数据段地址

MOV DS, AX

INC COUNTER ; COUNTER+=1

MOV SI, OFFSET INTMSG

CALL DPSTRI

MOV AL, 20H

OUT 20H, AL ; 向20H端口发送EOI命令

POP DS

POP BX

POP AX

IRET

NEW08H ENDP

DPSTRI PROC

PUSH AX

PUSH BX

DPS1:

LODSB ; 从DS:SI指向的内存读取一个字符到AL, SI+=1

CMP AL, 0 ; 不是'0', 就还没结束

JZ DPS2 ; 是'0', 就结束了

MOV BX, 0 ; BIOS显示功能页面号

MOV AH, 0EH ; BIOS显示字符功能号

INT 10H ; 调用BIOS视频中断

JMP DPS1

DPS2:

POP BX

POP AX

RET

DPSTRI ENDP

驻留中断服务程序

驻留TSR程序:

- 用户程序运行后仍保存在主存中,可供其它程序使用
- 利用DOS功能调用31H代替4CH终止程序
- 小型驻留程序常编写成COM程序,驻留程序也可编成EXE程序
- 需要驻留内存的程序段要写在前面

例. 系统08H号中断服务程序调用1CH中断,

每隔55ms调用这个报时中断,中断65543次就是过了1小时.

本例编写一个驻留内存的1CH内部中断服务程序.

每过1小时就显示信息,执行此程序后,报时中断服务程序将驻留内存

HOUR = 65543

HOURL = HOUR / 65536 ; 中断次数低16位

HOURH = HOUR MOD 65536 ; 中断次数高16位

NEWICH PROC

STI ; 开中断

PUSH SI ; "保护"

PUSH DS ; "保护"

MOV SI, CS

MOV DS, SI ; 所需数据在代码段中

ADD COUNTL, 1 ; 中断次数加1

ADC COUNTH, 0

CMP COUNTL, HOURL

JNZ NICH1 ; 未到1小时

CMP COUNTH, HOURH

JNZ NICH1 ; 未到1小时

MOV COUNTL, 0 ; 计数器清零

MOV COUNTH, 0

MOV SI, OFFSET INTMSG ; 使SI指向显示信息 (指向字符串的段内偏移)

CALL DPSTRI ; 调用显示字符串程序

NICH1:

POP DS

POP SI

IRET

COUNTL DW 0 ; 中断计数器低16位

COUNTH DW 0 ; 中断计数器高16位

INTMSG DB 'One Hour Has Passed!'

DB 0DH, 0AH, 0

NEWICH ENDP

DPSTRI PROC

PUSH AX

PUSH BX

DPSTRI:

LODSB ; 从DS:SI 读取一个字符到AL,同时SI加1

CMP AL, 0 ; 检查是否为字符串结束标志

JZ DPSTRI

MOV BX, 0

MOV AH, 0EH

INT 10H

JMP DPSTRI

POP BX

POP AX

RET

DPSTRI ENDP

START:

MOV AX, CS ; 使DS指向代码段

MOV DS, AX

MOV DX, OFFSET NEWICH ; DX = 新中断服务程序的偏移地址

INT 1CH PROGRAM

CLI ; 关

MOV AX, 251CH ; 设置中断向量, DB 0DH, 0AH, 'S'

INT 21H ; 号是1CH

STI ; 开

MOV DX, OFFSET TSRMSG

MOV AH, 09H ; 显示字符串

INT 21H

MOV DX, OFFSET START

ADD DX, 15

MOV CL, 4

SHR DX, CL

ADD DX, 10H

MOV AX, 3100H ; 程序驻留退出: AH=31H 返回码: AL=00H

INT 21H